

AI PROJECT — FINAL REPORT

Group Members: Arda Tutmaz (212134) & Güney Yılmaz (212132)
Course/Subject: Control Systems / Machine Learning for Control Problems
Topic: Linear Approximator for Control Problems using Temporal Difference Learning

Current Stage: Final Presentation & Evaluation

1. Executive Summary

This project designs and implements an autonomous parallel parking controller for a passenger vehicle using Reinforcement Learning (RL) combined with Linear Function Approximation. The car starts from five different initial positions on a two-lane street and must navigate into a narrow parking pocket (8.6m width, 3.0m depth) without colliding with the street boundaries, the curb, or neighboring zones.

By employing Tile Coding, Temporal Difference (TD) Q-Learning, and Potential-Based Reward Shaping, the vehicle learns a highly reliable, collision-free parking policy that successfully parallel parks the car parallel to the curb and deep inside the spot across all 5 starting states.

2. System Architecture & Environment Model

The physical dimensions and vehicle kinematics are modeled as follows:

Vehicle Dimensions: Length (L_c) = 5.1m, Width (W_c) = 2.3m.

Control Actions: A discrete action space containing 7 actions:

Action	Velocity (V)	Steering Angle (θ)
Stop	0 m/s	0
Reverse-Left	-2.0 m/s	$-\pi/4$
Reverse-Straight	-2.0 m/s	0
Reverse-Right	-2.0 m/s	$+\pi/4$
Forward-Left	+2.0 m/s	$-\pi/4$
Forward-Straight	+2.0 m/s	0
Forward-Right	+2.0 m/s	$+\pi/4$

State Space: Continuous 3-dimensional state representation: $s = (x, y, \alpha)$ where (x, y) is the geometric center of the car and α is the orientation angle.

3. Methodology & Algorithms

3.1. State Representation: Tile Coding

To handle the continuous state space (x, y, α) with standard RL algorithms, we use linear feature extraction via Tile Coding. The continuous space is partitioned using 8 overlapping, offset multi-dimensional grids (tilings). The bin sizes for each tiling are optimized at:

- X-axis bins: 35 (resolution of $\sim 0.76\text{m}$)
- Y-axis bins: 20 (resolution of $\sim 0.58\text{m}$)
- α -axis (Angle) bins: 20 (resolution of ~ 18 degrees)

This yields $8 \times 35 \times 20 \times 20 = 112,000$ total tiles. The state is represented as a sparse binary feature vector $x(s) \in \{0,1\}^{112,000}$ where exactly 8 features are active (equal to 1) at any given time.

3.2. Linear Function Approximation & Q-Learning

The action-value function $Q(s, a)$ is approximated linearly by multiplying the active features with a weight matrix W :

$$Q(s, a) = \sum W[i, a] \quad (\text{sum over active tiles})$$

We train the weights using one-step tabular-like Q-learning (Temporal Difference Control). The weight update rule operates via the LMS (Least Mean Squares) / Widrow-Hoff learning rule:

$$W[i, a] \leftarrow W[i, a] + (\alpha lr / N) \times (Y - Q(s, a))$$

where $N = 8$ (number of tilings), αlr is the learning rate, and Y is the TD Target: $Y = R + \gamma \times \max_{a'} Q(s', a')$

3.3. Exploration vs. Exploitation

An ϵ -greedy policy is used to balance exploration and exploitation. Epsilon (ϵ) starts at 0.5 and decays exponentially to a minimum of 0.01 as training progresses, allowing the policy to stabilize.

4. Key Implementation Innovations

4.1. Additive-Multiplicative Potential-Based Reward Shaping

To guide the agent from far away without distorting the optimal policy, we utilize Potential-Based Reward Shaping. We designed a custom potential function $\Phi(s)$ that decouples street navigation from parking pocket alignment:

$$\Phi(s) = 350.0 \times \text{pot_dist}(s) + 250.0 \times \text{pot_dist}(s) \times \text{pot_angle}(s)$$

where:

- $\text{pot_dist}(s) = \max(0.0, 1.0 - \text{dist}(s, \text{target}) / 15.0)$
- $\text{pot_angle}(s) = \max(0.0, 1.0 - \alpha_{\text{error}} / (\pi/2))$

Rationale: By multiplying the angle potential with the distance potential, the alignment penalty is scaled down when the car is far away on the street. This permits the car to turn freely to navigate towards the spot. As the car gets closer, the penalty for misalignment grows rapidly, forcing it to straighten up and align parallel to the curb.

4.2. Constrained Stopping (Gated Stop Action)

To ensure the vehicle parks deep inside the spot and parallel to the curb, we constrain the ϵ -greedy policy by blocking the Stop action unless the vehicle enters a defined rectangular safe target zone:

$$\text{Stop is allowed} \Leftrightarrow |x| < 0.8m \wedge y \in [-0.7, 0.25]m \wedge \alpha_{\text{error}} < 0.25 \text{ rad}$$

If these constraints are not met, the Q-value of the Stop action is set to a highly negative value (-9999.0), forcing the controller to continue adjusting its position.

5. Training and Results

Training Duration: The model is trained for 60,000 episodes with warm-start capabilities enabled (resuming training from weights.npy if available).

Validation Performance: The validation reward converges towards the optimal value (> -0.3).

Trajectory Verification:

- Episodes 1 & 3: The vehicle reaches the target center and stops successfully under 80 steps.
- Episodes 2, 4 & 5: The vehicle successfully navigates the street, performs correct reversing maneuvers, and fits itself completely parallel and deep inside the pocket without collisions.

6. Final Deliverables & Submission

All project requirements have been completed. The following final deliverables are prepared for submission:

- Complete Python source code (park_train.py, parking_model.py)
- Trained model weights file (weights.npy)
- This final project report